



Алгоритмы обработки больших данных

L06. State and Chekpoint, State Backends. Table API и SQL API

Апрель, 2026

Смирнов Сергей

smirnov.work@mail.ru

[@serg_v_smirnov](https://twitter.com/serg_v_smirnov)

Agenda

1. BigData intro
2. Stream & flink intro
3. Batch mode, pyflink runtime architecture
4. DataStream operators API, keyed state
5. Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks
- 6. *State and Chekpoint, State Backends. Table API и SQL API***
7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

Homework



Написать генератор сообщений (раз в секунду)
от IoT устройств вида:

- Тип устройства (id, идентификатор)
- Время события
- Температура
- Влажность

Генератор
сообщений IoT

Эти события публикуем
в топик кафки

kafka

Разработать ddl dml скрипты
для создания и наполнения
справочника типов IoT
устройств

- Id идентификатор типа
- TypeName наименование типа

pg

ddl.sql

dml.sql

src: kafka

src: pg

Рассчитываем среднюю температуру и
медиану влажности в каждой минуте

join

window

sink: kafka

Соединяем события из
кафки со статичным
справочником из постгри

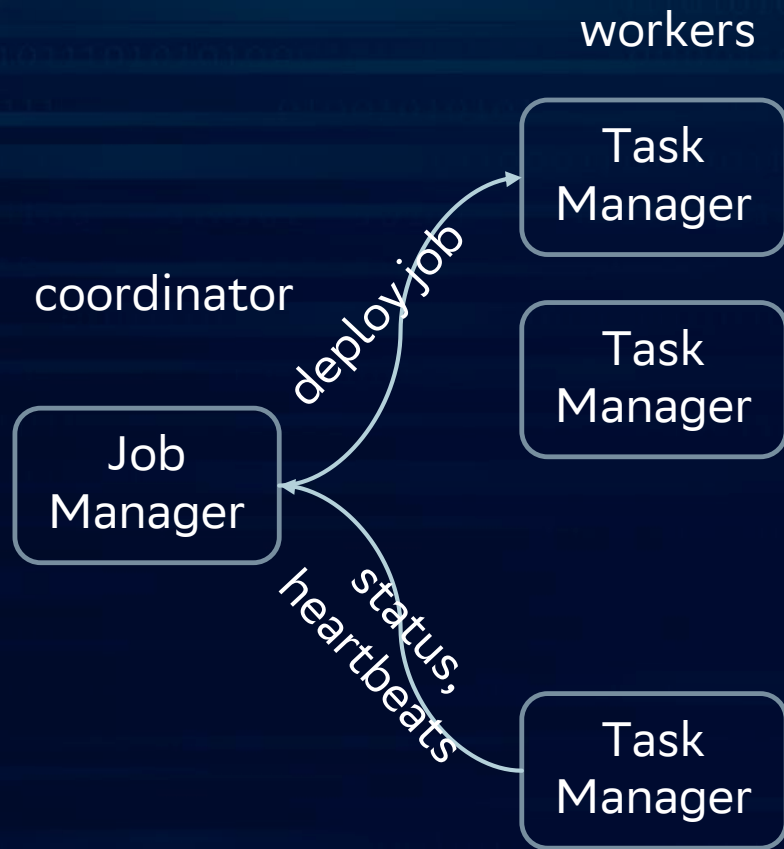
flink, работа в event time

Сохраняем полученный результат
в топик кафки в формате

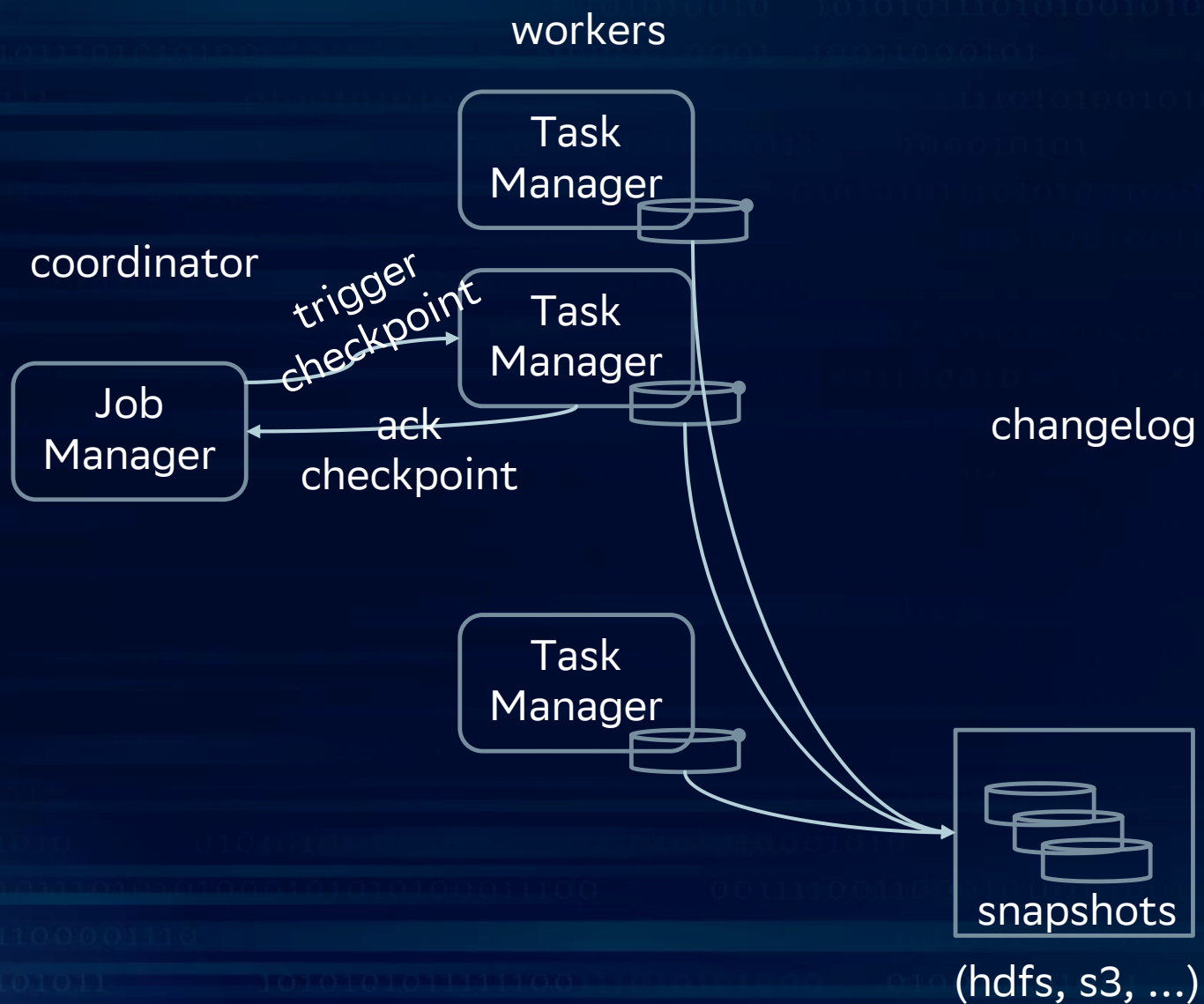
- Время (hh:mm)
- Тип устройства (из pg)
- Средняя температура
- Медиана влажности

kafka

Cluster components



Snapshot



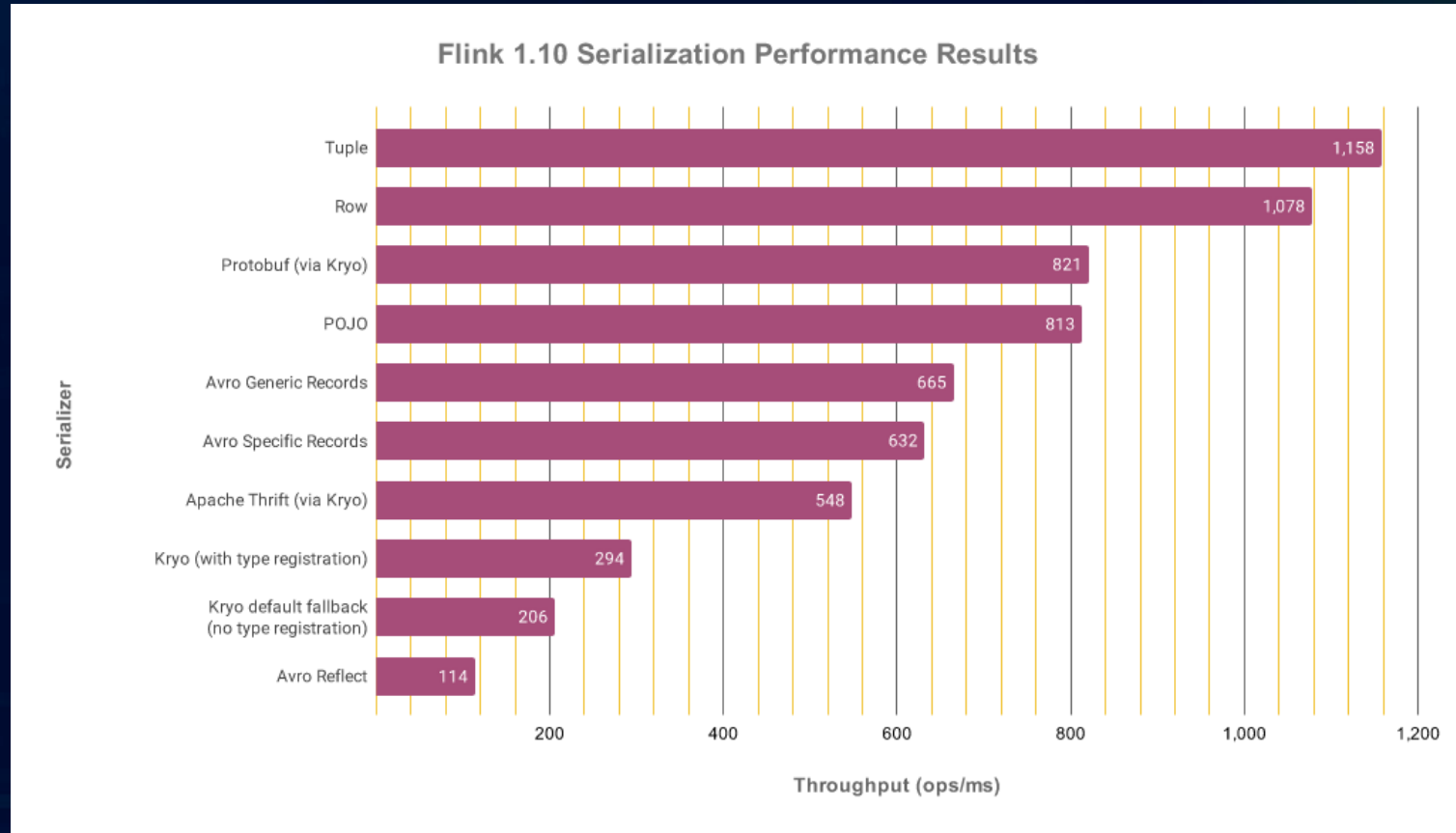
Serialization

Native support types

- Primitive types
- POJO
- Tuples, Scala Case Classes

Unsupported types

- Kryo by default
- User specified



<https://flink.apache.org/2020/04/15/flink-serialization-tuning-vol.-1-choosing-your-serializer-if-you-can/>

States



Keyed state
(datastream.keyBy)



Operator State
(non keyed state)

Not supported in python api, java only



Broadcast State
(special type of operator state)

Keyed state

Доступ через RuntimeContext

ValueState<T> getState(ValueStateDescriptor<T>)

ReducingState<T> getReducingState(ReducingStateDescriptor<T>)

ListState<T> getListState(ListStateDescriptor<T>)

AggregatingState<IN, OUT> getAggregatingState(AggregatingStateDescriptor<IN, ACC, OUT>)

MapState<UK, UV> getMapState(MapStateDescriptor<UK, UV>)

```
def open(self, runtime_context: RuntimeContext):  
    # Определяем state descriptor  
    state_descriptor = ValueStateDescriptor(  
        "avrg_cnt",  
        Types.TUPLE([Types.LONG(), Types.LONG()])  
    )  
    # Поднимаем состояние из стейта  
    self.state = runtime_context.get_state(state_descriptor)
```

TTL

```
def open(self, runtime_context: RuntimeContext):
    # Настройка TTL: состояние удаляется через 100 секунд после последнего обновления
    ttl_config = StateTtlConfig \
        .new_builder(Time.seconds(100)) \
        .set_update_type(StateTtlConfig.UpdateType.OnReadAndWrite) \
        .set_state_visibility(StateTtlConfig.StateVisibility.NeverReturnExpired) \
        .build()

    # Определяем state descriptor
    state_descriptor = ValueStateDescriptor(
        "avrg_cnt",
        Types.TUPLE([Types.LONG(), Types.LONG()])
    )

    #Включаем ttl
    state_descriptor.enable_time_to_live(ttl_config)
    # Поднимаем состояние из стейта
    self.state = runtime_context.get_state(state_descriptor)
```

Non-keyed state

implement CheckpointedFunction interface

```
void snapshotState(FunctionSnapshotContext context) throws Exception;  
void initializeState(FunctionInitializationContext context) throws Exception;
```

only list elements

```
private transient ListState<> opState;
```

Типовые решения

source & sink functions

Broadcast

Типовые операции

Соединение на словарь

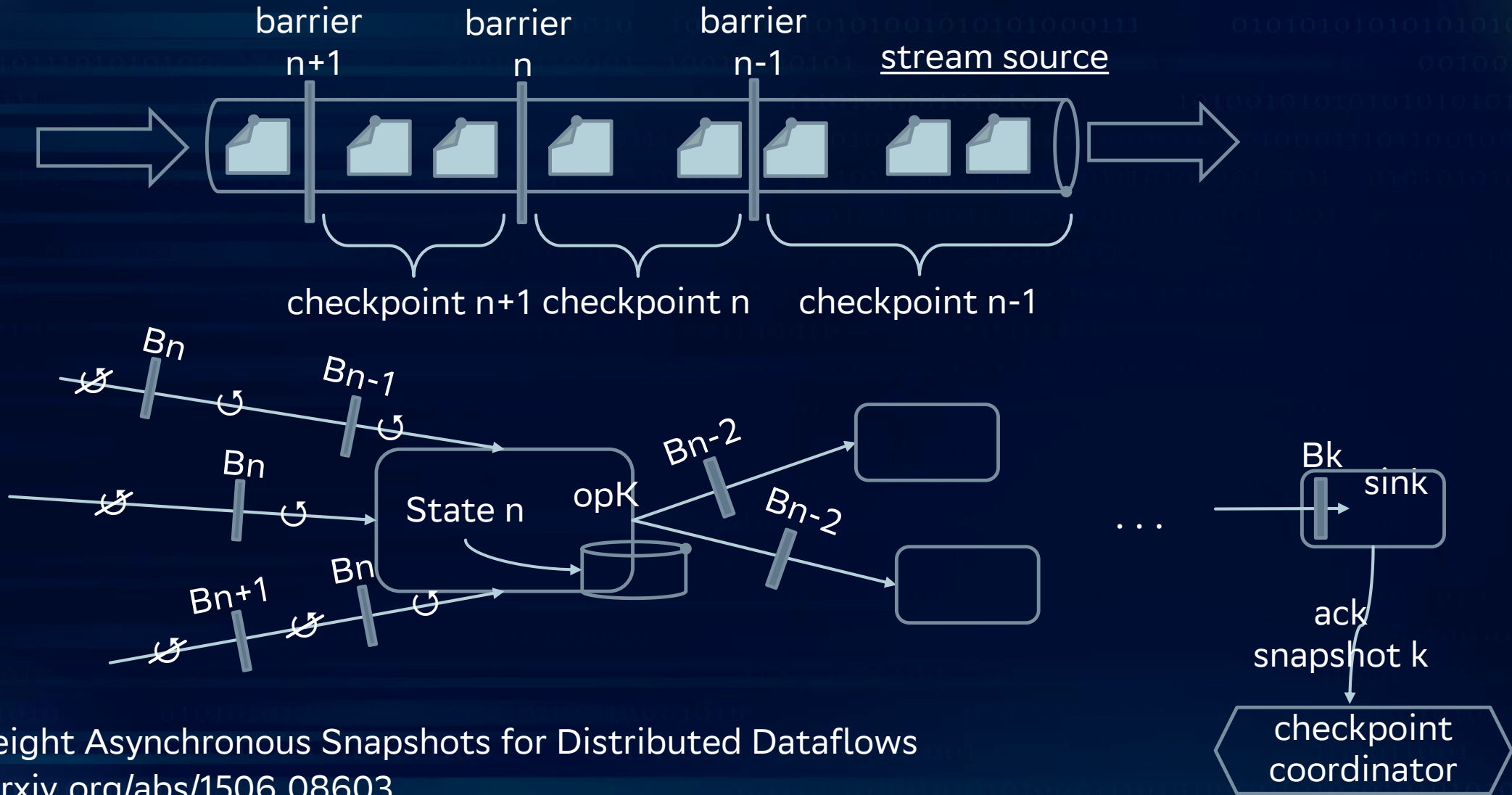
Применение правил

Распространение конфигурации

```
man_desc = MapStateDescriptor(  
    "dictBroadcast", # State name  
    Types.INT(), # Key type  
    Types.TUPLE([Types.STRING(), Types.INT()]) # value type  
)  
ds_dict = ds_man.broadcast(man_desc)  
  
ds.connect(ds_dict).process(  
    ManBroadcast(man_desc),  
    output_type=Types.PICKLED_BYTE_ARRAY()  
).print()
```

Checkpoint barriers

Alignment and unalignment checkpoints



Lightweight Asynchronous Snapshots for Distributed Dataflows

<http://arxiv.org/abs/1506.08603>

Inspired by Chandy-Lamport algorithm

<http://research.microsoft.com/en-us/um/people/lamport/pubs/chandy.pdf>

Operator checkpoints mechanics

- Получает от одного из входящих потоков барьер_n
- Прекращается обработка всех других сообщений из этого потока лежащих за барьером_n
- Аналогично для всех других входящих потоков, получение и обработка соответствующих сообщений только до барьера_n
- Как только получены все барьер_n из всех входящих потоков
 - Завершение всей внутренней обработки и публикация дальше по графу всех исходящих сообщений
 - Публикация барьер_n во все исходящие потоки
 - Создание snapshot на основе alignment state

unalignment checkpoints

- Смотрим барьер_n во входящих очередях (input network buffer)
- Перенаправляет барьер_n в конец всех исходящих потоков (output network buffer)
- Создает snapshot

+быстро

-дубли при восстановлении

-дополнительный размер на network buffers

checkpoints vs savepoints

Operation	Canonical Savepoint	Native Savepoint	Aligned Checkpoint	Unaligned Checkpoint
State backend change	✓	x	x	x
State Processor API (writing)	✓	x	x	x
State Processor API (reading)	✓	!	!	x
Self-contained and relocatable	✓	✓	x	x
Schema evolution	✓	!	!	!
Arbitrary job upgrade	✓	✓	✓	x
Non-arbitrary job upgrade	✓	✓	✓	✓
Flink minor version upgrade	✓	✓	✓	x
Flink bug/patch version upgrade	✓	✓	✓	✓
Rescaling	✓	✓	✓	✓

savepoints ~ user checkpoints (manually triggered)

✓ - Flink fully support this type of the snapshot

x - Flink doesn't support this type of the snapshot

! - While these operations currently work, Flink doesn't officially guarantee support for them, so there is a certain level of risk associated with them

https://nightlies.apache.org/flink/flink-docs-master/docs/ops/state/checkpoints_vs_savepoints/

Batch mode

Failure recovery for batch programs does not use checkpointing.

It is important to remember that because there are no checkpoints, certain features such as `CheckpointListener` and, as a result, Kafka's `EXACTLY_ONCE` mode or File Sink's `OnCheckpointRollingPolicy` won't work. If you need a transactional sink that works in `BATCH` mode make sure it uses the Unified Sink API as proposed in FLIP-143.

You can still use all the state primitives, it's just that the mechanism used for failure recovery will be different.

You should not cache the last seen watermark within an operator. In `BATCH` mode we process records key by key. As a result, the Watermark will switch from `MAX_VALUE` to `MIN_VALUE` between each key. You should not assume that the Watermark will always be ascending in an operator. For the same reasons timers will fire first in key order and then in timestamp order within each key. Moreover, operations that change a key manually are not supported.

https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/dev/datastream/execution_mode/

State backend

Local state

Memory

RocksDB

ForSt *flink 2.**

Snapshot fs

HDFS

Alluxio

XtreemFS

Amazon S3

Aliyun Object Storage Service

Azure Data Lake Store Gen2

Azure Blob Storage

Google Cloud Storage

Serialization

Primitive types – default flink

POJO – kryo (avro?)

other - kryo

Table, first step

```
settings = EnvironmentSettings.new_instance().in_batch_mode().build()
tenv = TableEnvironment.create(settings)
# В отличие от java нужно определять схему явно
# Без schema flink теряет данные или типы при работе с объектами Row
# В данном примере возвращает null, данные не проходят сериализацию
schema = DataTypes.ROW([
    DataTypes.FIELD("model", DataTypes.STRING()),
    DataTypes.FIELD("manufactureId", DataTypes.INT()),
    DataTypes.FIELD("year", DataTypes.INT())])
# Набор данных типа Row
data = [
    Row("Note", 2, 2021),
    Row("Lachetti", 1, 2008),
    Row("Almera", 2, 2018),
    Row("Civic", 3, 2015),
    Row("Accord", 3, 2021),
    Row("Captivea", 1, 2014),
    Row("Touareg", 4, 2016),
    Row("Lanos", 1, 2018),
    Row("Golf", 4, 2012)]
# Правильный способ: создание таблицы с привязкой к схеме
src = tenv.from_elements(data, schema)
# Вывод структуры таблицы (имена колонок и их типы) в консоль
src.print_schema()
```

Лаконичность

Выразительность

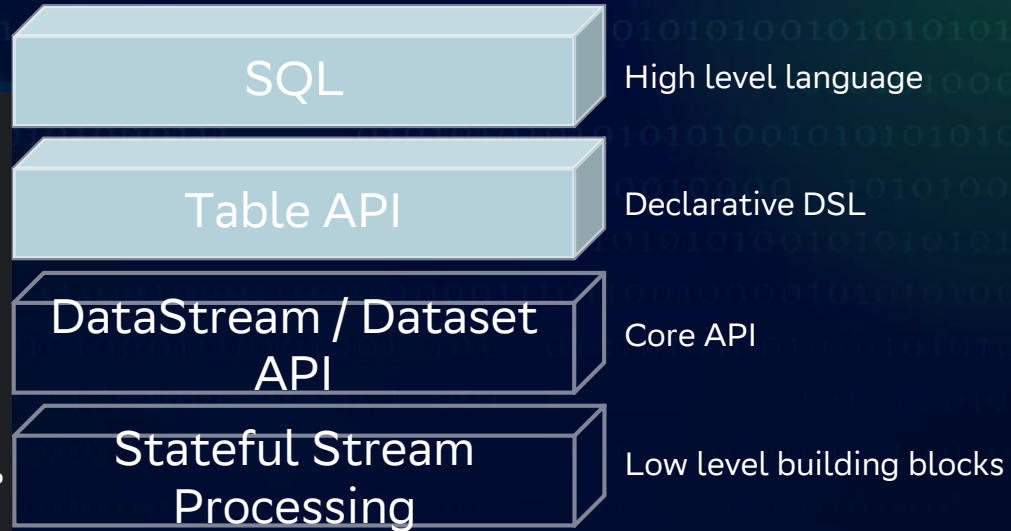


Table Environment

Окружение (TableEnvironment)

- Каталог таблиц (Table) с их метаданными
- Таблица всегда привязана к определенному окружению
- Невозможно одновременно использовать таблицы из разных окружений
- Для доступа к API необходимо использовать StreamTableEnvironment или TableEnvironment окружение

Table kinds

- Regular table (TABLE). Описание внешнего источника данных (файл, топик в кафке, таблица в БД)
- Virtual table (VIEW) – результат операций над TABLE

Table types

- Permanent table – требуют внешний каталог, видны из любых сессий
- Temporary table – используем их, если не оговорено иначе, видимость в пределах сессии

SQL

```
env = StreamExecutionEnvironment.get_execution_environment(conf)
env.add_jars(KAFKA_CONNECTOR)
tenv = StreamTableEnvironment.create(env)kafka_source = KafkaSource.builder() \ ...
ds_cars = env.from_source(kafka_source, WatermarkStrategy.no_watermarks(), "kafka src") \
    .map(parse_car, output_type=Types.TUPLE([Types.STRING(), Types.INT(), Types.INT(), Types.INT()])) \
    .filter(lambda i: i is not None and i[3] is not None)
t_cars = tenv.from_data_stream(
    ds_cars,
    Schema.new_builder()
        .column("f0", DataTypes.STRING()) # model
        .column("f1", DataTypes.INT())    # year
        .column("f2", DataTypes.INT())    # manufactureId
        .build()
).alias("model", "manufactureId", "year2")
tenv.create_temporary_view("cars", t_cars)
tenv.sql_query("select * from cars").execute().print()
```

Table ↔ DataStream

DataStream to Table

Table res = tEnv.fromDataStream(dsStream, ...)

Schema extraction: field position, field naming

Table to DataStream

Append Mode: только в режиме вставки. полученные сообщения нельзя изменять

DataStream<> ds = tEnv.toAppendStream(tab, ...)

Retract Mode: возможны операции добавления и удаления. размечается соответствующим флагом

DataStream<Tuple2<Boolean, Car>> ds = tEnv.toRetractStream(tab, Car.class);

Supported types: *Row, POJO, Case Class, Tuple, Atomic Type*

```
ds = env.from_collection(collection=data, type_info=type_info)
# Регистрируем DataStream как временное представление (view) в каталоге flink
t_env.create_temporary_view("users", ds)
# Выполняем sql запрос
res_tab = t_env.sql_query(
    "select concat(cast(id as string), '.', name) as name, age + 1 as age from users where age > 25"
)
# Используем to_data_stream для преобразования обратно в объект DataStream
# Важно: Updating tables are not supported by this method and will produce an exception
res_ds = t_env.to_data_stream(res_tab)res_ds.print()
# sql с агрегацией (создаем обновляемую таблицу)
# Считаем количество людей с одинаковым возрастом
res_tab = t_env.sql_query("""
    select
        age,
        count(id) as cnt
    from users
    group by age
""")
# Changelog DataStream
# Обычный to_data_stream не сработает для агрегации, в этом случае используем to_changelog_stream
res_ds = t_env.to_changelog_stream(res_tab)res_ds.print()
```

Window, table api

ProcessTime:

add column with proctime(), use .proctime() expression

```
schema = Schema.new_builder() \
    .column("f0", "STRING") \
    .column("f1", "INT") \
    .column("f2", "INT") \
    .column_by_expression("ts2", "PROCTIME()") \
    .build()
t_cars = tenv.from_data_stream(ds_cars, schema).alias("model", "manufactureId", "year", "ts2")
res = t_cars \
    .window(Tumble.over(expr.lit(5).seconds).on(expr.col("ts2")).alias("w")) \
    .group_by(expr.col("model"), expr.col("w")) \
    .select(
        expr.col("model"),
        # Технические поля окна: начало, конец и время обработки
        expr.col("w").start,
        expr.col("w").end,
        expr.col("w").proctime,
        # Агрегирующая функция (сумма по полю year)
        expr.col("year").sum.alias("sum_year")
    )
res.execute().print()
```

EventTime:

add watermarks, assign timestamp,

add column with rowtime(), use .rowtime() expression

```
t_cars = tenv.from_data_stream(
    ds_cars,
    Schema.new_builder()
        .column("f0", DataTypes.STRING())
        .column("f1", DataTypes.INT())
        .column("f2", DataTypes.INT())
        .column("f3", DataTypes.INT())
        # Для корректной работы TO_TIMESTAMP_LTZ превращаем секунды в миллисекунды
        .column_by_expression("ts_event", "TO_TIMESTAMP_LTZ(CAST(f3 AS BIGINT)*1000, 3)")
        .watermark("ts_event", "ts_event - INTERVAL '0' SECOND")
        .build()).alias("model", "man_id", "year", "raw_ts", "ts_event")
res = t_cars \
    .window(Tumble.over(lit(5).seconds).on(col("ts_event")).alias("w")) \
    .group_by(col("model"), col("w")) \
    .select(
        col("model"),
        col("w").start.alias("window_start"),
        col("w").end.alias("window_end"),
        col("year").avg.alias("avg_year")
    )
res.execute().print()
```

SQL, sources

```
CREATE TABLE rnd_dat (  
  random_int int,  
  random_str string,  
  ts as localtimeStamp,  
  WATERMARK FOR ts AS ts  
) WITH (  
  'connector' = 'custom_connector',  
  'rps'='5',  
  'rnd_min'='0',  
  'rnd_max'='99'  
)
```

```
CREATE TABLE click_stream (  
  user_id bigint,  
  category_id bigint,  
  place_id bigint,  
  data string,  
  ts timestamp(3),  
  proctime AS proctime(), -- generates processing-time  
  WATERMARK FOR ts AS ts - INTERVAL '10' SECOND --  
  watermark on ts, event time column  
) WITH (  
  'connector' = 'kafka', -- kafka connector  
  'topic' = 'clickstream', -- topic  
  'scan.startup.mode' = 'earliest-offset', -- from beginning  
  'properties.bootstrap.servers' = 'kafka:9094', -- broker  
  address  
  'format' = 'avro' -- avro data format  
)
```

Table connectors

```
CREATE TABLE hTable (  
  rowkey INT,  
  family1 ROW<q1 INT>,  
  family2 ROW<q2 STRING, q3 BIGINT>,  
  family3 ROW<q4 DOUBLE, q5 BOOLEAN, q6 STRING>,  
  PRIMARY KEY (rowkey) NOT ENFORCED  
) WITH (  
  'connector' = 'hbase-1.4',  
  'table-name' = 'mytable',  
  'zookeeper.quorum' = 'localhost:2181'  
);  
  
-- use ROW(...) construction function construct column families  
--and write data into the HBase table.  
-- assuming the schema of "T" is [rowkey, f1q1, f2q2, f2q3, f3q4, f3q5, f3q6]  
INSERT INTO hTable  
SELECT rowkey, ROW(f1q1), ROW(f2q2, f2q3), ROW(f3q4, f3q5, f3q6) FROM T;  
  
SELECT rowkey, family1, family3.q4, family3.q6 FROM hTable;  
  
SELECT * FROM myTopic  
LEFT JOIN hTable FOR SYSTEM_TIME AS OF myTopic.proctime  
ON myTopic.key = hTable.rowkey;
```

Name	Version	Source	Sink
Filesystem		Bounded and Unbounded Scan	Streaming Sink, Batch Sink
Elasticsearch	6.x & 7.x	Not supported	Streaming Sink, Batch Sink
Opensearch	1.x & 2.x	Not supported	Streaming Sink, Batch Sink
Apache Kafka	0.10+	Unbounded Scan	Streaming Sink, Batch Sink
Amazon DynamoDB		Not supported	Streaming Sink, Batch Sink
Amazon Kinesis Data Streams		Unbounded Scan	Streaming Sink
Amazon Kinesis Data Firehose		Not supported	Streaming Sink
JDBC		Bounded Scan, Lookup	Streaming Sink, Batch Sink
Apache HBase	1.4.x & 2.2.x	Bounded Scan, Lookup	Streaming Sink, Batch Sink
Apache Hive	Supported Versions	Unbounded Scan, Bounded Scan, Lookup	Streaming Sink, Batch Sink
MongoDB		Bounded Scan, Lookup	Streaming Sink, Batch Sink

SQL

Windows

TUMBLE
HOP
CUMULATE

Hints

```
/*+ OPTIONS('csv.ignore-parse-errors'='true') */  
/*+ OPTIONS('sink.partitionner'='round-robin') */  
/*+ OPTIONS('scan.startup.mode'='earliest-offset') */  
/*+ BROADCAST(t1) */  
/*+ SHUFFLE_HASH(t1) */  
/*+ SHUFFLE_MERGE(t1) */  
/*+ NEST_LOOP(t1, t3) */  
/*+ LOOKUP(...) */
```

```
tenv.sql_query("select TUMBLE_START(c.ts2, INTERVAL '5' SECOND), c.model, count(*), avg(year2) \n" +  
    "from cars c GROUP BY TUMBLE(c.ts2, INTERVAL '5' SECOND), c.model").execute().print()
```

```
tenv.sql_query("select /*+ LOOKUP('table'='tab2', 'async'='true',  
    'output-mode'='allow_unordered', 'capacity'='1000', 'timeout'='30s') */  
from tab1 as t1 join tab2 as t2 on t1.id = t2.id;").execute().print();
```

SQL

Query

with clause
select, where, order by
group by
group by window
agg(col) over
top n, limit
joins
match_recognize

Statement

SELECT, USE
CREATE, DROP, ALTER
INSERT, UPDATE, DELETE
ANALYZE, DESCRIBE, EXPLAIN, SHOW
LOAD, UNLOAD

Functions



System (built-in)

User-defined functions (UDF)

- Scalar: из скалярной величины в скалярную
- Table: набор скалярных величин в строку
- Aggregate: набор скалярных величин из нескольких строк в скаляр
- Table aggregate: набор скалярных величин из нескольких строк в новую строку (строки)
- Async table: lookup из источника

```
@udf(result_type=DataTypes.STRING())  
def my_upper_func(s):  
    return s.upper() if s is not None else None
```

```
tenv.create_temporary_view("cars", t_cars)
```

```
tenv.from_path("cars") \  
    .select(my_upper_func(col("model"))) \  
    .execute().print()
```

```
tenv.create_temporary_system_function("MyUpper", my_upper_func)  
tenv.from_path("cars") \  
    .select(call("MyUpper", col("model"))) \  
    .execute().print()
```

```
tenv.sql_query("select MyUpper(model) AS MODEL from cars") \  
    .execute().print()
```

Performance

```
select a.*, b.*  
from a join b on (...)  
where a.field=...
```

что выполнится
первым
join или filter

Заключительный проект



Написать генератор сообщений (раз в секунду)
от IoT устройств вида:

- Тип устройства (id, идентификатор)
- Время события
- Температура
- Влажность

Генератор
сообщений IoT

Эти события публикуем
в топик кафки

kafka

Разработать ddl dml скрипты
для создания и наполнения
справочника типов IoT
устройств

- Id идентификатор типа
- TypeName наименование типа

pg

ddl.sql

dml.sql

src: kafka

src: pg

Рассчитываем среднюю температуру и
медиану влажности в каждой минуте

join

window

sink: kafka

Соединяем события из
кафки со статичным
справочником из постгри

flink, работа в event time

Сохраняем полученный результат
в топик кафки в формате

- Время (hh:mm)
- Тип устройства (из pg)
- Средняя температура
- Медиана влажности

kafka

- Реализовать источник (source) и или
получатель (sink) на sql/table api
- Реализовать переход между
datastream и sql/table api

Questions



Спасибо за внимание